

Security And Access Document

Authentication, roles, row-level security, errors, and launch edge cases

Prepared from the Stratedge codebase

Last updated: 2026-06-13

Edgecipline Security And Access Document

Last updated: 2026-06-13

1. Purpose

This document explains how security and access should work for Edgecipline in plain English. It is written for a founder, product owner, or early engineering team that needs to understand who can access what, how user data is protected, what errors should look like, and what edge cases must be handled before launch.

Edgecipline stores sensitive user data:

- Personal account details.
- Trading history.
- Trading screenshots.
- Payment and subscription records.
- Psychology and behavioral trading notes.
- Admin actions and feedback.

The security goal is simple:

Every user should only see and change their own trading data, admins should have controlled operational access, and sensitive failures should be handled without leaking private information.

2. Recommended Authentication Method

Best Fit For This App

Use a hybrid authentication model:

- Email/password login for normal accounts.
- Google sign-in through Firebase for users who prefer OAuth.
- Backend-issued short-lived JWT access tokens for API requests.
- Backend-issued rotating refresh tokens stored as secure httpOnly cookies.
- Separate admin authentication using a different admin JWT secret.

This is the best fit because Edgeline is both a web app and mobile-wrapped app. Users need convenient login, while the backend still needs strong control over access to trade data, uploads, payments, and admin tools.

How Login Should Work

1. User logs in with email/password or Google.
2. Backend verifies the login.
3. Backend returns a short-lived access token.
4. Backend sets a long-lived refresh token in an httpOnly cookie.
5. Frontend sends the access token with API calls.
6. When the access token expires, the frontend silently refreshes it using the refresh cookie.
7. Refresh tokens rotate every time they are used.
8. If a reused refresh token is detected, the whole session family is revoked.

Why This Is Secure

- Access tokens expire quickly, so stolen tokens have a short lifetime.
- Refresh tokens are stored hashed in the database, not as raw tokens.
- Refresh tokens are kept in httpOnly cookies, so normal frontend JavaScript cannot read them.
- Token rotation detects replay attacks.
- tokenVersion lets the backend invalidate all active sessions after password reset or logout-all.
- Admin tokens use a different secret, so user tokens cannot accidentally become admin tokens.

3. Password And Account Security Rules

Password Requirements

Passwords should require:

- At least 8 characters.
- At least one uppercase letter.
- At least one lowercase letter.
- At least one number.
- At least one special character.

Login Protection

After repeated failed login attempts:

- Lock the account temporarily.
- Return a neutral error message.
- Do not reveal whether the email exists.

Current recommended rule:

- 5 failed attempts locks the account for 15 minutes.

Password Reset Protection

Password reset should use OTP with these rules:

- Store only a hashed OTP in the database.
- OTP expires after 10 minutes.
- Limit failed OTP attempts.
- After OTP verification, issue a short-lived reset token.
- Clear the OTP immediately after successful verification.
- After password reset, revoke all refresh tokens and increment tokenVersion.

Google Login Rules

- Only accept verified Google emails.
- Only accept Google as the sign-in provider.
- Do not silently convert password accounts into Google accounts.
- If an email is already registered with password login, Google login should be rejected with a clear message.

4. User Roles

Edgecipline should have two primary roles at launch:

- User
- Admin

Do not add more roles until there is a real operational need.

5. Role Permissions

Role: User

A regular user is a trader using Edgecipline.

Users can:

- Register and log in.
- Log out from current device.
- Log out from all devices.
- Accept terms and privacy policy.
- View their own profile.
- Update their own onboarding/preferences.
- Upload their own trade screenshots.
- View their own OCR job status.
- Confirm their own extracted trades.
- Manually create their own trades.
- View, edit, and delete their own Forex trades.
- View, edit, and delete their own Indian market trades.
- View their own dashboard.
- View their own analytics.
- Create, update, and delete their own setup strategies.
- Save their own checklist tracking records.
- View their own weekly reports.
- Generate their own weekly reports.
- View their own notifications.
- Register their own device token.
- Update their own notification preferences.
- Submit feedback.
- Create payment orders for their own account.
- Verify their own payment.

Users cannot:

- View another user's trades.
- Edit another user's trades.
- Delete another user's trades.
- View another user's screenshots.
- View another user's analytics.
- View another user's weekly reports.
- View another user's payment records.
- Change their own subscription status directly.
- Change their own role.
- Access admin routes.
- Access raw database IDs unless those IDs belong to their own records.
- Bypass terms acceptance for normal app usage.
- Upload unlimited screenshots after free limits unless subscribed.

Role: Admin

An admin is an internal operator or founder supporting users.

Admins can:

- Log in through the admin login flow.
- View platform-level admin dashboard.
- View user list.
- View user details needed for support.
- Activate, extend, or expire subscriptions.
- View payment history.
- View trade records for support/debugging.
- View feedback.
- Update feedback status.
- Send or manage operational notifications.
- Review platform analytics.
- Perform controlled administrative actions.

Admins cannot:

- Use normal user tokens to access admin routes.
- Access admin routes without the separate admin JWT secret.
- See raw passwords.
- See raw refresh tokens.
- See raw OTP values.
- Use admin tools as a substitute for direct database editing without audit.
- Delete user data permanently without a deliberate policy.
- Disable security checks for convenience.

Future Role: Support Agent

Do not build this in version one unless needed.

When added, support agents should be weaker than admins:

- Can view user account/subscription status.
- Can view support tickets/feedback.
- Can trigger password reset help instructions.
- Cannot view full trade screenshots by default.
- Cannot change roles.
- Cannot export full user data.
- Cannot perform destructive actions.

Future Role: Coach/Mentor

Do not build this in version one unless you launch a coaching product.

Coach access should require explicit user consent:

- User invites a coach.
- Coach can view selected journal/analytics data.
- Coach cannot edit trades.
- Coach cannot view payments.
- Coach cannot access auth/security settings.
- User can revoke access anytime.

6. Row-Level Security Rules

MongoDB does not automatically enforce row-level security like some SQL databases. In Edgecipline, row-level security must be enforced in the application layer.

Plain English rule:

Every query for user-owned data must include the logged-in user's ID.

Global Rule

If a collection has a user, userId, or owner field, every normal user request must be filtered by that owner field.

Examples:

- A user requesting trades must only query trades where `trade.user = currentUser.id`.
- A user requesting reports must only query reports where `report.user = currentUser.id`.
- A user requesting notification history must only query records where `notification.user = currentUser.id`.

User Collection Rules

Users can:

- Read their own basic profile.
- Update allowed preference fields.
- Accept terms for their own account.

Users cannot:

- List all users.
- Read another user's profile.
- Update role, subscription, total paid, token version, or login lock fields.

Admins can:

- List users.
- View user support fields.
- Update subscription-related fields.

Admins should not:

- Read password hashes.
- Read reset OTP hashes unless there is a specific debugging reason.
- Manually edit auth security fields except through controlled flows.

Trade Collection Rules

Applies to Trade and IndianTrade.

Users can:

- Create trades with `user = currentUser.id`.
- Read trades where `user = currentUser.id`.
- Update trades where `user = currentUser.id`.

- Soft-delete trades where user = currentUser.id.

Users cannot:

- Pass another user's ID in the request body.
- Change the user field after creation.
- Read soft-deleted trades unless the product explicitly supports trash/history.
- Access trades by ID unless that trade also belongs to them.

Admins can:

- View trades for support and moderation.
- Soft-delete or restore trades only through admin routes.

OCRJob Rules

Users can:

- Create OCR jobs for themselves.
- Poll OCR jobs where user = currentUser.id.
- Cancel OCR jobs where user = currentUser.id.
- Confirm OCR jobs where user = currentUser.id.

Users cannot:

- Confirm another user's OCR job.
- Use an OCR job to create a trade under another user.
- Access raw OCR/AI responses from another user's screenshot.

Admins can:

- Review OCR failures for support.
- Inspect extraction quality where necessary.

SetupStrategy Rules

Users can:

- Create setup strategies for themselves.
- Read their own setup strategies.
- Update their own setup strategies.
- Delete their own setup strategies.

Users cannot:

- Duplicate setup names within the same market.
- Access setup strategies owned by another user.
- Attach another user's setup strategy to their trade.

ChecklistTracking Rules

Users can:

- Save their own checklist completion records.
- Read their own checklist history.

Users cannot:

- Change checklist records for another user.

WeeklyReport Rules

Users can:

- Generate reports for their own trades.
- Read their own reports.

Users cannot:

- Generate reports using another user's trades.
- Read another user's report snapshots or AI feedback.

Payment Rules

Users can:

- Create payment orders for their own account.
- Verify payments for their own account.
- View their own subscription status.

Users cannot:

- Mark a payment as completed manually.
- Change subscriptionStatus.
- Change subscriptionExpiry.
- View another user's payments.
- Reuse another user's Razorpay order ID.

Admins can:

- View payment records.
- Manually activate subscriptions when needed.
- Add notes to payment records.

Notification Rules

Users can:

- View their own notifications.
- Mark their own notifications as read.
- Update their own notification preferences.
- Register their own device tokens.

Users cannot:

- Register a device token for another user.
- Read another user's notification history.
- Send system-wide notifications.

Admins can:

- Send operational notifications.
- View delivery status.

Feedback Rules

Users can:

- Submit feedback for themselves.
- View their own feedback if the product exposes that.

Users cannot:

- View all feedback.
- Change feedback status.
- Read another user's feedback.

Admins can:

- View feedback.
- Update feedback status.
- Add admin notes.

7. API Access Rules

Public Routes

These can be accessed without a valid user session:

- Register.
- Login.
- Google login.
- Forgot password.
- Verify OTP.
- Reset password.
- Refresh token.
- Logout.
- Health check.

Public does not mean unprotected. These routes still need validation and rate limits.

Protected User Routes

These require a valid user access token:

- Trades.
- Indian trades.
- Uploads.
- OCR status.
- Analytics.
- Dashboard.
- Setups.
- Checklists.
- Reports.
- Payments.
- Notifications.
- Profile.
- Feedback submission.

Terms-Gated Routes

After login, users who have not accepted the current terms should only access:

- Current profile route.
- Accept terms route.
- Logout route.

All normal product routes should return:

- HTTP 403
- Error code TERMS_NOT_ACCEPTED

Admin Routes

All admin routes must require:

- Admin token signed with admin secret.
- User exists.
- role = admin.
- Token version still valid.

Admin routes should live under:

- /api/admin/*

8. Error Handling Guide

Error messages should be useful to legitimate users but not helpful to attackers.

Standard Error Shape

All API errors should return:

```
{  
  "status": "error",  
  "message": "Human-readable message",  
  "errorCode": "MACHINE_READABLE_CODE"  
}
```

In production, server errors should always return:

```
{  
  "status": "error",  
  "message": "Something went wrong",  
  "errorCode": "INTERNAL_ERROR"  
}
```

Do not expose stack traces in production.

9. Major Failure Points And Correct Responses

Missing Login Token

When it happens:

- User calls protected route without a token.

Response:

- HTTP 401
- Error code AUTH_REQUIRED
- Message: No token provided

User experience:

- Send user to login.

Expired Access Token

When it happens:

- Access token has expired.

Response:

- HTTP 401
- Error code TOKEN_EXPIRED

User experience:

- Frontend should attempt silent refresh.
- If refresh fails, send user to login.

Invalid Or Tampered Token

When it happens:

- Token signature is invalid.
- Token is malformed.
- Token uses wrong secret.

Response:

- HTTP 401
- Error code INVALID_TOKEN

User experience:

- Clear local session and send user to login.

Token Version Mismatch

When it happens:

- User changed password.
- User logged out all devices.
- Admin/security flow invalidated tokens.

Response:

- HTTP 401
- Error code TOKEN_INVALIDATED

User experience:

- Clear session and ask user to log in again.

Refresh Token Missing

When it happens:

- Cookie expired.
- Browser blocked cookie.
- User cleared cookies.
- Mobile app did not send cookie.

Response:

- HTTP 401
- Error code AUTH_REQUIRED

User experience:

- Send user to login.

Refresh Token Reuse Detected

When it happens:

- A revoked refresh token is used again outside the short race grace window.
- This may indicate stolen session token.

Response:

- HTTP 401
- Error code TOKEN_REPLAY_DETECTED

Security action:

- Revoke the whole refresh token family.

User experience:

- Force login again.
- Optionally show: For your security, please sign in again.

Refresh Token Race

When it happens:

- Multiple tabs refresh at nearly the same time.

Response:

- HTTP 409
- Error code REFRESH_TOKEN_RACE

User experience:

- Retry refresh once after a short delay.
- Do not log user out immediately.

Terms Not Accepted

When it happens:

- User is authenticated but has not accepted current terms/privacy version.

Response:

- HTTP 403
- Error code TERMS_NOT_ACCEPTED

User experience:

- Send user to accept terms page.

Weak Password

When it happens:

- Password does not meet complexity rules.

Response:

- HTTP 400
- Error code WEAK_PASSWORD

User experience:

- Show exact missing requirement.

Account Locked

When it happens:

- Too many failed login attempts.

Response:

- HTTP 429
- Error code ACCOUNT_LOCKED

User experience:

- Tell user how long to wait.

Invalid Credentials

When it happens:

- Wrong email/password.

Response:

- HTTP 401
- Error code INVALID_CREDENTIALS

User experience:

- Show: Invalid email or password.

Security note:

- Do not reveal whether the email exists.

Google Auth Failure

When it happens:

- Google/Firebase token is invalid.
- Email is not verified.
- Provider is not Google.
- Google verification service times out.

Response:

- HTTP 401, 502, or 504
- Error codes: AUTH_FAILED, GOOGLE_AUTH_UNAVAILABLE, GOOGLE_AUTH_TIMEOUT

User experience:

- Ask user to try again or use email/password if that account was created locally.

Auth Provider Conflict

When it happens:

- User registered with password but tries Google login with same email.

Response:

- HTTP 409
- Error code AUTH_PROVIDER_CONFLICT

User experience:

- Tell user to log in with password.

Invalid Input Structure

When it happens:

- Request body contains keys starting with \$ or containing ..
- This can indicate NoSQL injection.

Response:

- HTTP 400
- Error code VALIDATION_ERROR

User experience:

- Show generic invalid input message.

Invalid Object ID

When it happens:

- User passes malformed MongoDB ID.

Response:

- HTTP 400
- Error code INVALID_ID

User experience:

- Show resource not found or invalid request, depending on route.

Security note:

- For records that may belong to another user, prefer 404 Not found after ownership check instead of revealing the record exists.

Duplicate Resource

When it happens:

- Duplicate email.
- Duplicate setup strategy name.
- Duplicate transaction ID.
- Duplicate notification dedupe key.

Response:

- HTTP 409
- Error code DUPLICATE_RESOURCE

User experience:

- Tell user what needs to be changed.

File Too Large

When it happens:

- Screenshot exceeds upload limit.

Response:

- HTTP 400
- Error code LIMIT_FILE_SIZE

User experience:

- Ask user to upload a smaller screenshot.

Invalid File Type

When it happens:

- User uploads a non-image or unsupported image.

Response:

- HTTP 400
- Error code INVALID_FILE_TYPE

User experience:

- Ask user to upload a supported screenshot image.

Non-Trade Image Uploaded

When it happens:

- OCR/AI determines image is not a trade screenshot.

Response:

- HTTP 400
 - Error code such as INVALID_TRADE_SCREENSHOT
- Security/product action:
- Reject extraction.
 - Delete image from Cloudinary when possible.
 - Do not save a completed trade.

OCR Or AI Provider Failure

When it happens:

- Google Vision fails.
- Tesseract fails.
- Gemini fails.
- Provider times out.

Response:

- Mark OCR job as FAILED.
- Return safe error to client.

User experience:

- Offer retry.
- Offer manual trade entry.

Security note:

- Do not expose raw provider error messages to users.

Payment Verification Failure

When it happens:

- Razorpay signature mismatch.
- Order ID does not belong to user.
- Payment already used.

Response:

- HTTP 400 or 403
- Error code such as PAYMENT_VERIFICATION_FAILED

Security action:

- Do not activate subscription.
- Log the attempt.

Rate Limit Exceeded

When it happens:

- Too many auth, upload, status, refresh, or global requests.

Response:

- HTTP 429
- Include Retry-After.

User experience:

- Ask user to wait and retry.

Database Unavailable

When it happens:

- MongoDB disconnected.

Response:

- HTTP 503 for health check.
- HTTP 500 for normal requests with generic message.

User experience:

- Show temporary service issue.

Redis Unavailable

When it happens:

- Rate limiter or queue/cache backend is unavailable.

Current recommendation:

- Rate limiter may fail open so normal users are not blocked.
- OCR upload queue should fail closed if jobs cannot be created.

User experience:

- Uploads may show temporary processing unavailable.

Admin Forbidden

When it happens:

- User token or non-admin role tries admin route.

Response:

- HTTP 403
- Error code FORBIDDEN

User experience:

- Do not show admin UI to non-admin users.

10. Logging Rules

Log enough to debug attacks and support issues, but never log secrets.

Safe to log:

- Route.
- Method.
- Status code.
- Duration.
- User ID.
- Error code.
- IP address.
- User agent.
- OCR job ID.
- Payment order ID.

Never log:

- Passwords.
- Raw refresh tokens.
- Access tokens.
- OTPs.
- Reset tokens.
- Razorpay secret.
- Firebase private key.
- Google Vision private key.
- Cloudinary secret.
- Full Authorization header.
- Full cookie header.

11. Launch Edge Cases To Handle

Authentication Edge Cases

- User opens app with expired access token.
- User opens app with missing refresh cookie.
- User has valid token but account was deleted.
- User has valid token but tokenVersion changed.
- Multiple browser tabs refresh at the same time.
- User resets password while another device is active.
- User logs out from all devices.
- User registered with Google tries password login.
- User registered with password tries Google login.
- Firebase is misconfigured.
- Google token verification times out.
- Mobile app requires SameSite=None; Secure=true refresh cookie.

Authorization Edge Cases

- User requests trade ID owned by another user.
- User tries to update user field on a trade.
- User tries to access another user's OCR job.
- User tries to confirm another user's OCR job.
- User tries to view another user's payment.
- User tries to update own role or subscription status.
- Non-admin accesses /api/admin/*.
- Admin token is signed with user secret by mistake.
- User token is accepted by admin middleware by mistake.

Upload And OCR Edge Cases

- Upload is too large.
- Upload is unsupported file type.
- File extension says image but content is not image.
- Screenshot is not a trade.
- Screenshot has multiple trades.
- Screenshot has low confidence extraction.
- Cloudinary upload succeeds but OCR job creation fails.
- OCR job succeeds but user never confirms.
- OCR job expires before confirmation.
- User cancels upload while worker is processing.
- Worker crashes mid-job.
- Duplicate status polling happens from multiple tabs.
- Gemini returns malformed JSON.
- OCR extracts wrong broker/instrument.
- Indian options screenshot has CE/PE ambiguity.
- Forex screenshot contains partial data.

Trade Data Edge Cases

- Negative profit.
- Zero profit.
- Missing exit price.
- Missing stop loss.
- Custom risk-reward value.
- Trade date before account creation.
- Future trade date.
- Duplicate trade upload.
- User deletes a trade used in analytics.
- Soft-deleted trades accidentally included in analytics.
- Indian option expiry stored as invalid date.
- Equity and option fields mixed together.

Payment Edge Cases

- User closes Razorpay checkout.
- Payment succeeds but verification request fails.
- Payment verification is retried.
- Same Razorpay payment ID submitted twice.
- Razorpay order belongs to another user.
- Subscription expiry calculation crosses month/year boundary.
- Admin manually activates subscription after failed payment.
- Payment record says completed but user subscription not updated.

Notification Edge Cases

- Device token belongs to another user.
- Device token expires.
- Push delivery fails repeatedly.
- User disables push notifications.
- User enables quiet hours.
- Duplicate notification would be sent.
- Morning mentor sends outside intended timezone.
- Notification deep link points to deleted trade/report.

Admin Edge Cases

- Admin account password reset invalidates sessions.
- Admin tries destructive action too many times.
- Admin edits subscription accidentally.
- Admin views sensitive screenshots without audit.
- Admin action fails halfway.
- Admin logs out but cookie remains stale.

Privacy And Compliance Edge Cases

- User asks to delete account.
- User asks to export data.
- User uploads screenshot containing account number or personal information.

- Logs accidentally contain OCR text with private data.
- Sentry captures sensitive request data.
- AI provider receives screenshot data.
- Terms/privacy version changes and users must re-accept.

12. Pre-Launch Security Checklist

- User and admin JWT secrets are different.
- JWT secrets are at least 32 characters.
- Refresh tokens are stored hashed only.
- Refresh cookie is httpOnly.
- Production cookies use secure settings.
- Capacitor app refresh works with correct cookie settings.
- Password reset revokes all sessions.
- Logout-all revokes all sessions.
- Terms gate works on all protected product pages.
- Every user-owned query filters by current user ID.
- Admin routes use admin middleware only.
- User routes do not accept admin-only fields from request body.
- Uploads validate real file type, not just extension.
- Non-trade images are rejected.
- Payment verification checks signature and ownership.
- Soft-deleted trades are excluded from analytics.
- Rate limits are enabled for auth, upload, refresh, and status polling.
- Sentry scrubs Authorization and cookies.
- Logs do not include secrets or tokens.
- Production CORS allowlist is explicit.
- Health endpoint does not expose secrets.
- .env files are not committed.
- Admin accounts are created intentionally and reviewed.
- Security tests cover IDOR and token replay.

13. Founder-Friendly Security Policy

For version one, use this policy:

1. Users own their data.
2. Every user-owned record must be queried with the logged-in user's ID.
3. Admins can support users, but admin access must be separate and limited.
4. Passwords, OTPs, and refresh tokens must never be stored or logged in raw form.
5. Payment status can only change after verified payment or explicit admin action.
6. AI/OCR failures should never create false completed trades.
7. When in doubt, fail safely and ask the user to retry or log the trade manually.